

COMP 3311

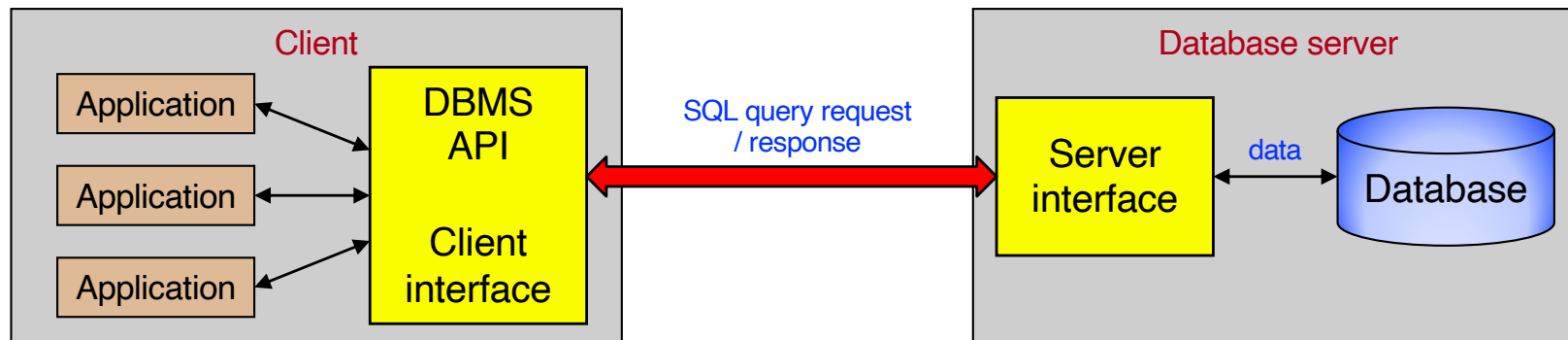
DATABASE MANAGEMENT SYSTEMS

LECTURE 11

STRUCTURED QUERY LANGUAGE (SQL): DATA MANIPULATION LANGUAGE (DML) & DATA DEFINITION LANGUAGE (DDL)

DBMS API

- To utilize DBMS services, client applications use a specific application programming interface (API) provided by the DBMS.
 - 👉 Facebook, Google, Instagram, etc. have such APIs.
- The **DBMS API** exposes an interface through which **the services provided by the DBMS** can be accessed from a programming language.
 - The client and server interfaces often are implemented in the form of network sockets that use a specific port number on the server (e.g., port 1521 for the CSE department's Oracle Database server).

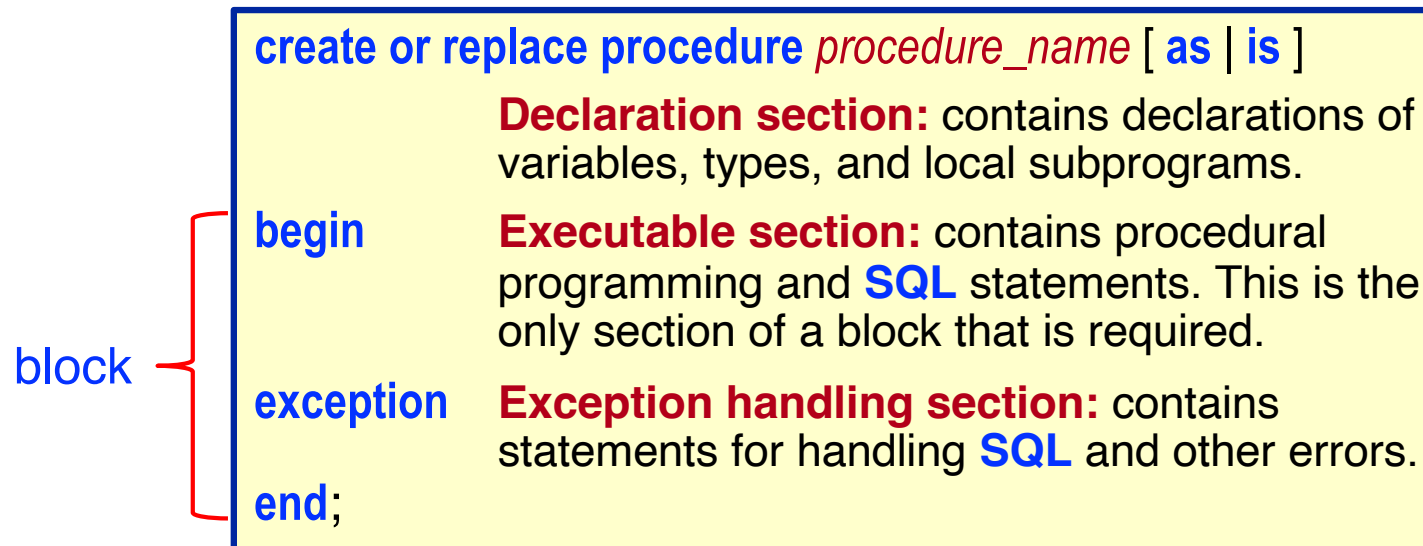


ORACLE PL/SQL

- Oracle **PL/SQL** (Procedural Language/SQL) is a **C**-like procedural programming language that allows **SQL** statements to be embedded into it.
 - 👉 Many programming languages (e.g., Java, C) have similar features.
- An Oracle **PL/SQL** program is stored as a **database object** in a database and can be a
 - **procedure** which does not return a value and is invoked using the **exec** keyword.
 - **function** which returns a value using the **return** keyword and is invoked by assigning its result to a variable or using it in a **select** statement.
- Both types of Oracle **PL/SQL** programs can accept parameters.

ORACLE PL/SQL: BASIC STRUCTURE

- A **block** is the basic processing unit; it is delimited by **begin...end** statements and can be nested (i.e., a **block** can be contained within a **block**).



Allowed – **SQL DML** statements: **select**, **insert**, **update**, **delete**

Not allowed – **SQL DDL** statements: **create**, **drop**, **alter**, **rename**

ORACLE PL/SQL:

DATA TYPES & CONTROL FLOW STATEMENTS

- The data type of a program variable can be the same type as:
 - the attributes of a relation (i.e., the basic types – **number**, **int**, **char**, **varchar**, etc.).
 - a table attribute (***table_name.attribute_name*%type**) or a row (***table_name*%rowtype**).

Sequential control

- goto** – branch to a label unconditionally
- null** – pass control to the next statement
- return** – return control to the calling block;
may return a value.

Conditional control

- if-then**, **if-then-else**, **if-then-elsif**
– conditional processing
- case** – selects one sequence of
statements to execute

Iterative control

- loop statements end loop;**
- while condition loop statements end loop;**
- for loop_variable in [reverse] lower_bound..upper_bound loop statements end loop;**
- exit / exit when condition** – exit the current loop possibly conditionally
- continue / continue when condition** – exit current loop iteration; continue with next iteration

SELECT INTO STATEMENT

```
select attribute_name,... into variable_name,... from table_name [where condition];
```

- Retrieves values from a relation in the database and assigns them to *one or more variables* in the **PL/SQL** program.
- The **select ... into** statement must retrieve only one record since a simple program variable normally can hold only one value.
 - If the **select ... into** statement retrieves more than one or no value, an exception is raised ➔ handled in the **exception** section.
- The number of attributes and their data type in the **select** clause must match with the number of variables and their data types in the **into** clause.
- The attribute values are retrieved, and the variables are populated, in the same order as specified in the **select** clause.

PL/SQL PROCEDURE EXAMPLE

Query: Increment the rating of a client if the rating is less than 5.

```
create or replace procedure Example1 (cid in int) as
```

```
  clientName Client.name%type;
```

```
  clientRating Client.rating%type;
```

```
begin
```

```
-- Fetch the client's name and rating into the program variables clientName and clientRating
```

```
select name, rating into clientName, clientRating from Client where clientId=cid;
```

```
if clientRating<5 then
```

```
  update Client set rating=clientRating+1 where clientId=cid;
```

```
-- Write client updated message to the Script Output tab
```

```
  database system_output.put_line('Client ' || clientName || '(' || cid || ') rating updated from ' ||  
    clientRating || ' to ' || (clientRating+1) || '.');
```

```
else
```

```
-- Write client NOT updated message to the Script Output tab
```

```
  database system_output.put_line('Client ' || clientName || '(' || cid || ') rating ' || clientRating || ' NOT updated.');
```

```
end if;
```

```
end Example1;
```

Program variables clientName and clientRating are of the **same type** as the name and rating attributes in the Client relation.

Must retrieve at most one record

Assigns name value to clientName variable and rating value to clientRating variable.

Client(clientId, name, hkid, address, district, rating)

COMP 3311

© 2026

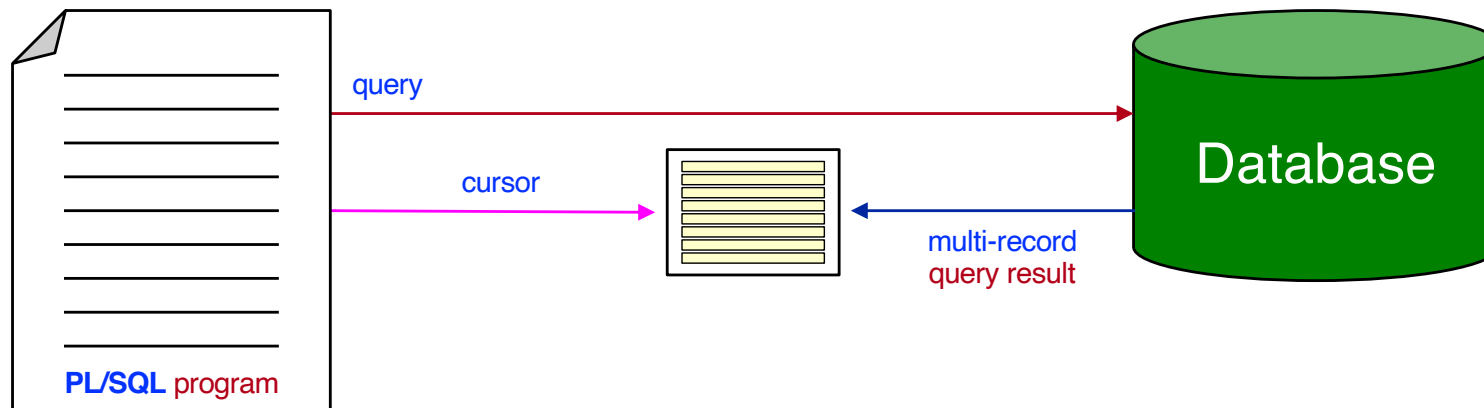


LECTURE 11
SQL DML & DDL

7

CURSORS

- Procedural programming languages such as **PL/SQL** normally **retrieve only one record at a time**.
- If a **select** statement returns more than one record, a **PL/SQL** program needs a way to **access the records one-at-a-time**.



- A **cursor** is **like a pointer** and **points to a single record** in a **query result** allowing access to the attribute values of that record.

👉 **A PL/SQL program uses a cursor to iterate over all records in a query result.**

CURSORS (CONT'D)

- A **PL/SQL** cursor is named and defined in the **Declaration section**

cursor *cursor_name* **is** *select_statement*;

and can be used and managed in a program

- *explicitly* using **open**, **fetch** and **close** statements and by checking cursor status.
- *implicitly* using the **for...loop** statement where *cursor_name* is used to iterate from the first cursor record to the last cursor record.

- When a cursor is managed explicitly, it's status can be queried to check various conditions that may arise.

cursor_name%**found** Returns TRUE if the fetch operation succeeded; else returns FALSE.

cursor_name%**notfound** Returns TRUE if the fetch operation failed; else returns FALSE.

cursor_name%**isopen** Returns TRUE if the cursor is still open; else returns FALSE.

cursor_name%**rowcount** Returns the number of records in the query result.

PL/SQL CURSOR EXAMPLE

Query: Determine which clients have a loan.

```
create or replace procedure Example2 as
currentClientId Client.clientId%type;
-- Declare the cursors for the Client and Borrower relations
cursor clientCursor is select * from Client order by name;
cursor borrowerCursor is select count(clientId) numLoans from Borrower where clientId=currentClientId;
begin
-- Fetch the clientCursor records one-by-one
for clientRecord in clientCursor loop
-- Assign the client id for the current Client record
currentClientId:=clientRecord.clientId;
-- Fetch the borrowerCursor records one-by-one
for borrowerRecord in borrowerCursor loop
-- Insert into appropriate relation
if borrowerRecord.numLoans=0 then
insert into LoanSummary values (clientRecord.clientId, clientRecord.name, 'No');
else
insert into LoanSummary values (clientRecord.clientId, clientRecord.name, 'Yes');
end if;
end loop;
end loop;
end Example2;
```

clientCursor and borrowerCursor define what data should be retrieved when their **select** statement is executed.

Executes clientCursor **select** statement (i.e., retrieves all Client records).

Assigns the **clientId** value in the current Client record pointed to by the cursor to the variable **currentClientId**.

Executes borrowerCursor **select** statement using the value in **currentClientId** to find matching Borrower records (i.e., retrieves only the Borrower records where **clientId** matches the value in **currentClientId**).

PL/SQL EXCEPTIONS

- Predefined exceptions are raised implicitly by **PL/SQL** if the exception occurs.
- A program also can define application specific exceptions in the **Declaration section**

exception_name **exception**;

raise them explicitly within a **begin...end** block

```
if condition then
    raise exception_name;
end if;
```

and handle them in the **Exception section** within the **begin...end** block.

```
exception
when exception_name then
    :
```

Predefined Exceptions

ACCESS_INTO_NULL	ORA-06530
CASE_NOT_FOUND	ORA-06592
COLLECTION_IS_NULL	ORA-06531
CURSOR_ALREADY_OPEN	ORA-06511
DUP_VAL_ON_INDEX	ORA-00001
INVALID_CURSOR	ORA-01001
INVALID_NUMBER	ORA-01722
LOGIN_DENIED	ORA-01017
NO_DATA_FOUND	ORA-01403
NOT_LOGGED_ON	ORA-01012
PROGRAM_ERROR	ORA-06501
ROWTYPE_MISMATCH	ORA-06504
SELF_IS_NULL	ORA-30625
STORAGE_ERROR	ORA-06500
SUBSCRIPT_BEYOND_COUNT	ORA-06533
SUBSCRIPT_OUTSIDE_LIMIT	ORA-06532
SYS_INVALID_ROWID	ORA-01410
TIMEOUT_ON_RESOURCE	ORA-00051
TOO_MANY_ROWS	ORA-01422
VALUE_ERROR	ORA-06502
ZERO_DIVIDE	ORA-01476



Query: Increment the rating of a client if the rating is less than 5.

PL/SQL EXCEPTIONS EXAMPLE

```
create or replace procedure Example3 (cid in int) as
-- clientName is the same type as name in the Client relation
clientName Client.name%type;
-- clientRating is the same type as rating in the Client relation
clientRating Client.rating%type;
begin
-- Fetch the client's name and rating into the variables clientName and clientRating
select name, rating into clientName, clientRating from Client where clientId=cid;
if clientRating<5 then
    update Client set rating=clientRating+1 where clientId=cid;
    -- Output client updated message
    dbms_output.put_line('Client ' || clientName || '(' || cid || ') rating updated from ' ||
        clientRating || ' to ' || (clientRating+1) || '.');
else
    -- Output client NOT updated message
    dbms_output.put_line('Client ' || clientName || '(' || cid || ') rating ' || clientRating || ' NOT updated.');
```

end if;

```
exception
when no_data_found then
    -- Output exception message
    dbms_output.put_line('There is no client with id ' || cid || '.');
```

end Example3;

This pre-defined exception is raised if the **select** statement does not return any record.

SQL DATA DEFINITION LANGUAGE (DDL)

The **SQL DDL** allows the specification of:

- The **schema** for a relation (i.e., relation name and attributes).
- The **type of values** associated with an attribute (i.e., the domain of values of the attribute, such as string, number, date, etc.).
- **Integrity constraints** (ICs).
 - domain, key, foreign key, general
- The **set of indexes** to be maintained for each relation.
- The **physical storage structure** of each relation on disk.
- **Security and authorization** information for each relation.

BASIC DOMAIN TYPES

char (<i>n</i>)	Fixed length character string with length <i>n</i> (padded to <i>n</i> with spaces).
varchar2 (<i>n</i>)	Variable-length character string with maximum length <i>n</i> . ¹
int	An integer (a finite subset of the integers that is machine-dependent).
smallint	A small integer (a machine-dependent subset of the integer domain type).
number (<i>p,d</i>)	A fixed point number with a total of <i>p</i> digits (the precision) and <i>d</i> digits to the right of the decimal point.
float (<i>n</i>)	Floating point number, with user-specified precision of at least <i>n</i> digits.
date	A date containing a (4 digit) year, month, day of month. ²
time	The time of day, in hours, minutes and seconds.
timestamp	A combination of date and time.

 **Some relational DBMSs also allow user-defined domain types.**

1. Oracle uses the nonstandard **varchar2** domain type for variable-length character strings. It also allows use of the **varchar** domain type which is the ANSI standard for variable length character strings. Currently **varchar** and **varchar2** are the same in Oracle. However, Oracle recommends to not use **varchar** as it may change its usage in future.

2. The Oracle date domain type contains the (4 digit) year, the month, the day, the hours, the minutes and the seconds.

CREATING RELATIONS

- The **create table** statement is used to define and create a relation.
- The **domain type** of each attribute needs to be specified.
 - A **default** value can be specified for an attribute (only used when no insert value is provided; attributes must be *explicitly* specified).
 - **Null values** are allowed in all the basic domain types.

```
create table Client (  
  clientId  int,  
  name      varchar2(15)  
  hkid      char(10),  
  address   varchar2(20),  
  district  varchar2(20));
```

```
create table Account (  
  accountNo  char(5),  
  balance    number(8,2) default 0,  
  branchName varchar2(15));
```

👉 **The domain type of an attribute is enforced by the DBMS whenever tuples are added or modified.**

```
create table Depositor (  
  clientId  int,  
  accountNo char(5));
```

ALTERING AND DESTROYING RELATIONS

- The **alter table** statement is used to **add** attributes to, **modify** attributes in or **drop** attributes from an existing relation.

```
alter table Client  
  add postalCode int;
```

The schema is altered by **adding a new attribute** and extending every tuple in the current relation instance with a **null value for the new attribute**.
(The keyword **column** is not allowed.)

```
alter table Client  
  drop column postalCode;
```

The schema is altered by **dropping the attribute** from the relation instance and **deleting its value in every tuple**.
(The keyword **column** is required.)

- The **drop table** statement deletes **all** information about a relation (both **data and schema**).

```
drop table Client;
```

INTEGRITY CONSTRAINTS (IC)

An integrity constraint (IC) ensures that authorized changes to the database do not result in a loss of data consistency.

👉 **An IC guards against accidental damage to the database.**

- ICs are **obtained from the requirements of the real-world application** that is being described in the database relations.
 - An integrity constraint is a statement about **all possible instances!**
 - We know from common knowledge that an attribute like **hkid** is a key, but in general the constraint that an attribute, such as **accountNo**, is a key must be given to us by the client.
- We can check a database instance to see if an IC is violated, but we can **never** infer an IC by looking at a database instance.

Why?

DOMAIN CONSTRAINTS

- Domain constraints define **valid values** for attributes and are used to **validate values** inserted into the database and **validate queries** to ensure that the comparisons make sense.
- Besides a basic domain type, additional constraints can be specified on attributes in the **create table** statement.

not null specifies that the **value** of an attribute **cannot be null**.

primary key specifies that the value of an attribute (or a set of attributes) is a key for a relation (since the value of a primary key **cannot be null** there is no need to specify **not null**).

unique specifies that the value of an attribute or a set of attributes is a candidate key (the value of a candidate key **can be null**).

foreign key specifies that the value of one or more attributes may reference the value of a primary key in another relation.

check specifies a predicate (condition) that the values of an attribute in every tuple of the relation must satisfy.

FOREIGN KEY CONSTRAINT

A **foreign key** is a set of attributes in one relation whose values **must** match the primary key values in another relation or be **null**.

✎ **A foreign key must reference the entire primary key of the referenced relation.**

Example: Only clients listed in the Client relation should be allowed to hold accounts (i.e., appear in the Depositor relation).

```
create table Client (
  clientId int primary key,
  name varchar2(15) not null,
  hkid char(10) not null,
  address varchar2(20) not null,
  district varchar2(20) not null,
  rating int not null,
  unique (hkid);
```

```
create table Depositor (
  clientId int,
  accountNo char(5),
  primary key (clientId, accountNo),
  foreign key (clientId) references Client(clientId));
```

Note: Should also have a foreign key constraint for accountNo.

✎ **Every clientId value in the Depositor relation must reference a tuple in the Client relation with a matching clientId value.**

✎ **A referenced relation must be created before the relations that reference it.**

FOREIGN KEY CONSTRAINT: ACTIONS

- What should be done if a **Depositor** tuple with a non-existent **accountNo** is inserted?

👉 **Reject it!**

- What should be done if an **Account** tuple is deleted?
 - Disallow deletion** of an **Account** tuple that is referred to by a **Depositor** tuple (*default action*).
 - Alternatively, delete all **Depositor** tuples that refer to it (*on delete cascade*).

Account			Depositor	
<u>accountNo</u>	balance	branchName	<u>clientId</u>	<u>accountNo</u>
A-117	35000	Star House	3	A-117
A-120	80000	Star House	3	A-120
A-215	55000	Diamond Hill	3	A-301
A-301	75000	Pacific Place	4	A-215
A-322	45000	Pacific Place	4	A-322
			8	A-120

FOREIGN KEY CONSTRAINT: ACTIONS (CONT'D)

- What should be done if a **Depositor** tuple with a non-existent **accountNo** is inserted?

👉 **Reject it!**

- What should be done if an **Account** tuple is deleted?
 1. **Disallow deletion** of an **Account** tuple that is referred to by a **Depositor** tuple (*default action*).
 2. Alternatively, delete all **Depositor** tuples that refer to it (*on delete cascade*).
 3. Set **accountNo** in **Depositor** tuples that refer to it to a *default value* (*on delete set default*).
 4. Set **accountNo** in **Depositor** tuples that refer to it to a *null value* (*on delete set null*).

👉 Note that 3 and 4 are not applicable in the example since **accountNo** is part of the primary key of **Depositor**.

FOREIGN KEY CONSTRAINT: ACTIONS (CONT'D)

- What should be done if the primary key **accountNo** of a tuple in **Account** is updated?

👉 **Reject it!**

- Alternatively, propagate the update to the tuples in the **Depositor** relation with matching account numbers (**on update cascade**).

<u>accountNo</u>	balance	branchName
A-117	35000	Star House
A-120	80000	Star House
A-215	55000	Diamond Hill
A-301	75000	Pacific Place
A-322	45000	Pacific Place

<u>clientId</u>	<u>accountNo</u>
3	A-117
3	A-120
3	A-301
4	A-215
4	A-322
8	A-120

FOREIGN KEY CONSTRAINT: ACTIONS (CONT'D)

- What should be done if the primary key **accountNo** of a tuple in **Account** is updated?

 **Reject it!**

- Alternatively, propagate the update to the tuples in the **Depositor** relation with matching account numbers (**on update cascade**).

```
create table Depositor (  
  clientId int,  
  accountNo char(5),  
  primary key (clientId, accountNo),  
  foreign key (accountNo) references Account(accountNo)  
  on delete cascade);
```

The referential integrity actions in the **referencing relation** (**Depositor**) are triggered when a tuple in the **referenced relation** (**Account**) is deleted or updated.

Oracle Note

Oracle does not support **on update cascade**.

CHECK CLAUSE: ATTRIBUTES

- The **check** clause is used to add an integrity constraint for an attribute and can contain an **arbitrary predicate** (i.e., **condition**).^{*}
- The predicate is specified in the definition of a relation and checked whenever there is an update to the relation.

Example: Ensure that a loan amount is greater than or equal to \$1,000 but less than or equal to \$100,000.

```
create table Loan (
  loanNo      char(5) primary key,
  amount      number(8,2) check (amount >= 1000 and amount <= 100000),
  year        char(4),
  branchName  varchar2(15) not null);
```

```
create table Loan (
  loanNo      char(5) primary key,
  amount      number(8,2) check (amount between 1000 and 100000),
  year        char(4),
  branchName  varchar2(15) not null);
```

Loan(loanNo, amount, year, branchName)

VIEWS

- A **view** is a relation created by a query that **provides a way to hide certain data** from users.
- The **create view** statement creates a view.
`create view view-name as <query expression>`
 where: <query expression> is any legal **SQL** query.

IMPORTANT NOTE

Views should never be used to express queries!

Example: Create a view of the Loan relation that hides the amount.

```
create view BranchLoan as
select loanNo, year, branchName
from Loan;
```

Loan

<u>loanNo</u>	amount	year	branchName
L-171	115000	2024	Star House
L-321	40000	2024	Pacific Place
L-434	52000	2025	Festival Walk
L-435	55000	2025	Festival Walk



BranchLoan

<u>loanNo</u>	year	branchName
L-171	2024	Star House
L-321	2024	Pacific Place
L-434	2025	Festival Walk
L-435	2025	Festival Walk

- The **drop view** statement deletes a view.

Example: Drop the BranchLoan view.

```
drop view BranchLoan;
```

Loan(loanNo, amount, year, *branchName*)



VIEWS (CONTD)

- A view can be queried the same as any other relation.

Query: Find all Star House branch loans using the BranchLoan view.

```
select loanNo
from BranchLoan
where branchName='Star House';
```

Query result

loanNo
L-171

BranchLoan

<u>loanNo</u>	year	branchName
L-171	2024	Star House
L-321	2024	Pacific Place
L-434	2025	Festival Walk
L-435	2025	Festival Walk

- A user who has access to the **BranchLoan** view, but not the **Loan** relation, cannot see the amount.
- Normally, a view is **stored as a query** and **produced on demand**; however, some relational DBMSs allow views to be **stored as relations** and to be **automatically kept up-to-date** if the relations that define the view change.

👉 **Such stored views are called materialized views.**

VIEW UPDATE

- Assume that we allow users who have access to the **BranchLoan** view to insert into the underlying relations using the view.

Example: Add a new tuple to the BranchLoan view.

```
insert into BranchLoan values ('L-180', '2025', 'Star House');
```

- This insertion must be represented by the insertion of the tuple ('L-180', null, '2025', 'Star House') into the **Loan** relation as shown below.

BranchLoan			Loan			
loanNo	year	branchName	loanNo	amount	year	branchName
L-171	2024	Star House	L-171	115000	2024	Star House
L-321	2024	Pacific Place	L-321	40000	2024	Pacific Place
L-434	2025	Festival Walk	L-434	52000	2025	Festival Walk
L-435	2025	Festival Walk	L-435	55000	2025	Festival Walk
L-180	2025	Star House	L-180	null	2025	Star House

✎ This insertion is allowed provided **amount** can be null.

BranchLoan(loanNo, year, *branchName*)

COMP 3311

© 2026



Loan(loanNo, amount, year, *branchName*)

LECTURE 11
SQL DML & DDL

27

VIEW UPDATE PROBLEMS

create view BranchBorrower as
select branchName, clientId
from Borrower natural join Loan;

Borrower		JOIN _{loanNo}	Loan				=	BranchBorrower	
<u>clientId</u>	<u>loanNo</u>		<u>loanNo</u>	amount	year	branchName		branchName	clientId
3	L-171		L-171	115000	2024	Star House		Star House	3
3	L-435		L-321	40000	2024	Pacific Place		Pacific Place	4
4	L-321		L-434	52000	2025	Festival Walk		Festival Walk	4
4	L-434		L-435	55000	2025	Festival Walk		Festival Walk	3
17	L-435							Festival Walk	17

Suppose we want to insert the tuple ('Diamond Hill', 7) into the BranchBorrower view as created above.

✎ The Loan and Borrower relations must be updated accordingly.

VIEW UPDATE PROBLEMS (CONTD)

- Inserting the tuple ('Diamond Hill', 7) into the BranchBorrower view requires the following inserts into Borrower and Loan.

```
insert into Borrower (clientId, loanNo) values (7, null);
insert into Loan (loanNo, amount, year, branchName)
values (null, null, null, 'Diamond Hill');
```

Borrower

<u>clientId</u>	<u>loanNo</u>
3	L-171
3	L-435
4	L-321
4	L-434
17	L-435
7	null

Loan

<u>loanNo</u>	amount	year	branchName
L-171	115000	2024	Star House
L-321	40000	2024	Pacific Place
L-434	52000	2025	Festival Walk
L-435	55000	2025	Festival Walk
null	null	null	Diamond Hill

✎ These updates cannot be performed because the value of loanNo, which is part of the primary key of Borrower as well as the primary key of Loan, is null!

✎ **Recall:** The entity integrity constraint requires that the value of the primary key attributes not be null!

VIEW UPDATE PROBLEMS (CONTD)

- Suppose these updates were allowed, would they have the desired effect (i.e., does the **BranchBorrower** view include the tuple ('Diamond Hill', 7)?

Borrower		Loan			
<u>clientId</u>	<u>loanNo</u>	<u>loanNo</u>	amount	year	branchName
3	L-171	L-171	115000	2024	Star House
3	L-435	L-321	40000	2024	Pacific Place
4	L-321	L-434	52000	2025	Festival Walk
4	L-434	L-435	55000	2025	Festival Walk
17	L-435	null	null	null	Diamond Hill
7	null				

JOIN_{loanNo}

No! Why?

✎ The new tuples cannot be joined on **loanNo** because it is null!

✎ Recall: null never matches anything.

RULES FOR UPDATABLE VIEWS

- A view is **updateable** (i.e., tuples can be inserted, updated and deleted) if all the following **conditions** are **satisfied** by the query that defines the view.
 - The **from** clause has **only one relation**.
 - The **select** clause contains **only attribute names of the relation**, and does not have any expressions, aggregates, or **distinct** specification.
 - Any attribute not listed in the **select** clause can be set to **null** (i.e., it does not have a **not null** constraint and is not part of a primary key.
 - The query does not have a **group by** or **having** clause.

 **The rules for view update are often DBMS dependent.**

Oracle Note
Oracle does not
support assertions.

ASSERTIONS

- An **assertion** is an arbitrary **SQL** predicate that the database must always satisfy.

- An assertion takes the form:

create assertion <assertion-name> **check** <predicate>

- Difference from previous constraints:
 - A **constraint** is associated with a **single relation** and **checked when there is an update on this specific relation**.
 - An **assertion** may be associated with **several relations** and is **checked every time there is an update anywhere**.

 **Assertion testing may introduce a significant amount of overhead and so should be used with great care.**

ASSERTION EXAMPLE

Example: The sum of all loan amounts for each branch must be less than the sum of all account balances at the branch.

```
create assertion loanSumConstraint as check
(not exists (select *
             from Branch
             where (select sum(amount)
                    from Loan natural join Branch)
                  >=
                  (select sum(balance)
                   from Account natural join Branch)));
```

- Since the assertion refers to more than one relation, it cannot be included as a constraint in the definition of the **Loan** or **Account** relations but must be stated as an assertion.

Loan(loanNo, amount, year, *branchName*)

Branch(branchName, district, liabilities, assets)

Account(accountNo, balance, *branchName*)

TRIGGERS

- A **trigger** is a statement that is **executed automatically** by the DBMS as a **side effect of a modification** to the database.
- A trigger must
 - specify the **condition(s)** under which it is to be executed.
 - specify the **action(s)** to be taken when it executes.
- A trigger is used to implement integrity constraints that cannot be specified by other **SQL** constraints.

Example: Automatically update the **liabilities** value of the **Branch** relation whenever a **balance** value of the **Account** relation is updated.

Account(accountNo, balance, branchName)
Branch(branchName, district, liabilities, assets)

TRIGGER EXAMPLE

- Suppose that instead of allowing negative account balances, a bank deals with overdrafts by
 - creating a loan in the amount of the overdraft.
 - giving this loan a loan number which is identical to the account number of the overdrawn account.
 - setting the account balance to zero.
- The **condition for executing the trigger** is an update to the account relation that results in a negative balance value.

TRIGGER EXAMPLE (CONT'D)

```
create or replace trigger overdraft
before update of balance on Account
for each row
when (new.balance<0)
declare
    currentYear    Loan.year%type;
begin
    select to_char(sysdate, 'YYYY') into currentYear from dual;
    insert into Loan values (:old.accountNo, - :new.balance,
                           currentYear, :old.branchName);
    insert into Borrower (select clientName, accountNo
                        from Depositor
                        where accountNo=:old.accountNo);
    :new.balance:=0);
end;
```

- The keywords **old** and **new** are used to access the value of **balance** before/after the update.

Account(accountNo, balance, branchName)

Loan(loanNo, amount, year, branchName)

Depositor(clientId, accountNo)

Borrower(clientId, loanNo)



ORACLE DATABASE: TRIGGER SYNTAX (SIMPLIFIED)

```
create [or replace] trigger trigger-name
    {before | after} {delete | insert | update [of column-name ...]} or ... on table-name
    [referencing {old as old | new as new} ...]
    [for each row]
    [when (condition)]
    PL/SQL_block
```

[] optional
{ } select one
... can repeat immediately
preceding construct

- **before** | **after** - specifies that the trigger should fire (execute) before/after executing the triggering event (i.e., before or after a **delete**, **insert** or **update** statement).
- **delete** | **insert** | **update of column-name** - specifies the statement that causes the trigger to fire; more than one of the listed statements can be specified (e.g., **before insert or after delete** ...).
- **on table-name** - specifies the relation on which to create the trigger.
- **referencing {old as old | new as new}** - specifies correlation names for old and new values of the current row. The default correlation names are old and new. (e.g., old.balance refers to the balance value before an update while new.balance refers to the balance value after an update). When using old and new in a PL/SQL_block they must be preceded by a colon (e.g., :old.balance).
- **for each row** - specifies that the trigger fires once for each row that is affected by the triggering statement and that meets the optional trigger constraint defined in the **when** clause. If this clause is omitted, then the trigger fires only once and not for each row.
- **when (condition)** - specifies the condition that must be satisfied for the trigger to fire.

STRUCTURED QUERY LANGUAGE (SQL): SUMMARY

- **Structured Query Language (SQL)** is a relational query language that provides facilities to

Query Relations

- **select-from-where** statement
- aggregate functions (avg, min, max, sum, count)
- **group by** with **having** clause
- nested subqueries (to test for set membership, comparison, cardinality)
- analytical functions
- access a database from a programming language

Create and Modify Relations

- create, alter, drop relations
- insert, delete, update tuples
- specify integrity constraints: domain, key, foreign key, general
- create views, assertions and triggers